Санкт-Петербургский государственный университет
Кафедра системного программирования

Разработка компилятора для защиты от атак на цепочки поставок

Балышев Артем Максимович, 2 курс Программной инженерии

Научный руководитель: инженер-исследователь, Косарев Дмитрий Сергеевич

Санкт-Петербург
2026

Область применения проекта и актуальность

Атака на цепочку поставок — ситуация, при которой злоумышленник внедряет уязвимость в систему через зависимости этой системы

- Например, через компилятор, которым эта система собирается

Полностью защититься от таких атак не удастся

- В силу того, что у любой программной системы есть зависимости, которым система не может доверять в полной мере
- Обеспечить определённый уровень доверия к системе всё же можно

Некоторые организации заинтересованы в том, чтобы создавать доверенные программные системы и, как следствие, вынуждены думать о том, как выявлять такие атаки

Концепция проекта

Предлагается разработать компилятор, который позволит порождать проверяемый код

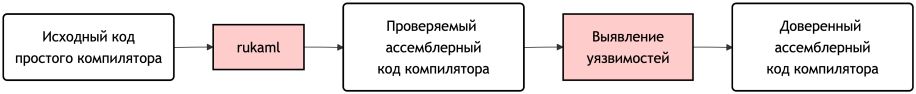
- Требуется определить критерии, согласно которым скомпилированная программа может считаться проверяемой
- Подразумевается, что компилятор будет способен собрать простые инструменты сборки
- После этого результаты компиляции могут быть проверены на отсутствие уязвимостей
- После проверки полученные артефакты позволят провести раскручивание (англ. bootstrapping) доверенной операционной системы

Проект rukaml и его архитектура

rukaml — компилятор функционального языка

- Разрабатывается на МатМехе СПбГУ, преимущественно в рамках учебных практик
- Фронтенд и генерация кода написаны на OCaml
- Стандартная библиотека написана на Си
- Позволяет порождать RISC-V 64 ASM, AMD64 ASM и LLVM IR

Предлагается развитие проекта в качестве инструмента, упрощающего построение доверенных цепочек сборки



Цель и задачи

Целью работы в рамках проекта rukaml является реализация поддержки языковых конструкций, необходимых для разработки компиляторов и других языковых инструментов, и обеспечение защиты собираемых программ от атак на цепочки поставок

Задачи:

- Реализация поддержки алгебраических типов данных и сопоставления выражений с образцом
- Расширение стандартной библиотеки примитивами форматированного вывода, работы с файловой системой и обработки аргументов командной строки
- Оптимизация размера порождаемого компилятором кода
- Обеспечение воспроизводимости сборок

Особенности реализации

- Алгебраические типы позволяют описывать промежуточные представления программ
  - Например, деревья абстрактного синтаксиса
- Сопоставление с образцом позволяет обрабатывать эти промежуточные представления
  - Например, выполнять проверку типов

Особенности реализации

Расширение стандартной библиотеки

- Примитивы форматированного вывода
  - Упрощает написание pretty-printer'ов
  - Упрощает написание модулей генерации кода
- Примитивы работы с файловой системой и обработки аргументов командной строки
  - Позволяют компилировать программы, работающие в режиме утилиты командной строки

Особенности реализации

- Оптимизация размера порождаемого кода
  - Выбрана как одна из основных метрик проверяемости результатов компиляции
  - Чем меньше кода генерирует компилятор, тем дешевле будет проверка этого кода
- Обеспечение воспроизводимости сборок
  - Выделено как одно из основных требований к доверенному компилятору
  - Потому что люди, занимающиеся верификацией, используют проверку воспроизводимости артефактов как один из способов выявления уязвимостей
  - rukaml использует систему сборки dune, которая уделяет большое внимание воспроизводимости сборок
  - Ожидается, что выбранная экосистема позволит достичь воспроизводимости сборок без необходимости вмешательства со стороны пользователя

Результаты

На сегодня:

- Реализована поддержка алгебраических типов и сопоставления с образцом
- Стандартная библиотека расширена ранее упомянутыми примитивами

Благодаря этому с помощью rukaml удалось собрать простой компилятор подмножества языка Си

- Обрабатывает аргументы командной строки
- Выполняет построение AST
- Печатает AST в человекочитаемом виде
- Выполняет трансляцию в RISC-V 64 Assembly

Дальнейшая работа

- Расширение возможностей компилируемого языка
- Оптимизация объема порождаемого кода
- Обеспечение воспроизводимости сборок
- Доверенный ассемблер и доверенный линкер
- Работа без libc и запуск на bare metal